Member 1 Briefing: Algorithm and Research Design

Begin here, even if you have never studied numerical methods. This guide builds the vocabulary before asking you to explain the equations.

Beginner foundation: the minimum math vocabulary

Matrix, vector, and equation system

A vector is a list of numbers, for example [2, 5, -1]. A matrix is a table of numbers. Multiplying a matrix by a vector creates a new vector. The project wants a vector x that makes:

```
A*x = b
```

true. This is called a linear system. It is just a compact way of writing many ordinary equations at once.

Sparse and dense

A dense matrix stores every cell. A sparse matrix has many zero cells. The matrices in this project are sparse, so we keep only their nonzero values. This makes the project suitable for large scientific problems.

Exact versus approximate

An exact answer has no error. Computers use finite number formats, so real projects normally calculate an approximate answer: one close enough for a defined accuracy requirement. The project never assumes an approximation is good; it measures its error before returning success.

The three stages in one simple analogy

Think of finding a destination with a poor map. SPAI makes the map easier to read, GMRES suggests a direction to walk, and iterative refinement checks where you are and asks for another direction if needed. In code terms: SPAI builds P, GMRES returns correction d, and refinement updates x=x+d.

Research words that can confuse people

- Algorithm: a repeatable sequence of steps for solving a problem.
- Preconditioner: a helper that changes a problem into an easier form.
- Residual: what remains wrong, b-A*x.
- Tolerance: a small threshold used to decide whether an error is small enough.
- FP64/FP32: computer number formats. FP64 keeps roughly 16 decimal digits; FP32 keeps roughly 7.
- BSPAI/mixed precision: research terms mentioned by the project literature. They are not claims about the active implementation.

Now read the rest of this guide as a second layer. It gives the equations, research framing, presentation script, and professor-question answers.

How to read the symbols

Read A*x=b aloud as: "matrix A multiplied by vector x gives vector b." Read r=b-A*x as: "residual r is what remains wrong." Read d as: "the correction we will add to x." You do not need to manipulate the equations in a viva; you need to explain the role of each symbol.

Advanced terms decoded before they appear

Word	Plain meaning
Inverse	A matrix that undoes another matrix. An exact inverse is usually too expensive to store.
Identity matrix	Ones on the main diagonal and zeros elsewhere; it leaves a vector unchanged.
Pattern	Positions where SPAI is currently allowed to place nonzero values.

QR - A stable internal fitting technique.

Krylov subspace - GMRES's growing collection of promising correction directions.

Orthogonal - Directions with no overlap; they add independent information.

Backward error - A scale-aware measure of how well the computed answer fits the original equation.

Your role in one sentence

Explain the numerical problem, the SPAI-GMRES-IR method, the research motivation, and the boundary between the intended research direction and the code that is actually implemented.

Project truth to memorize

This project solves a sparse real square linear system

$$Ax = b.$$

The active solver implements uniform-precision SPAI-GMRES-IR in two precisions (FP64 and FP32) and on CPU and CUDA backends. The CPU FP64 solver is the clearest reference path. It uses an adaptive sparse approximate inverse (SPAI) preconditioner, unrestarted left-preconditioned GMRES, and iterative refinement.

The project name and literature plan mention BSPAI and mixed precision, but the active code does not implement bucketed SPAI or a mixed-precision solver. Describe those only as research motivation or possible future work.

Why this problem is difficult

For a dense matrix, direct methods such as Gaussian elimination can be practical at small sizes. The matrices in this project are sparse: most entries are zero. Storing a dense inverse would usually destroy that sparsity and use too much memory. Iterative methods are attractive because they largely use sparse matrix-vector products. However, an iterative method can converge slowly when the matrix is poorly conditioned. A preconditioner changes the system into an equivalent form that is easier for the iterative method.

The three layers of the method

1. SPAI: construct a sparse approximate inverse

We seek a sparse matrix P that behaves approximately like A^{-1}:

$$P \approx A^{-1}, \qquad PA \approx I.$$

It is deliberately sparse, so it is inexpensive to store and apply. The code does not calculate the dense exact inverse.

The implementation first computes row scaling. For row k, it defines

$$d_k = \frac{1}{\max_j |a_{kj}|}, \qquad D = \mathrm{diag}(d_k),$$

then forms

$$B = A^T D.$$

It builds a sparse right approximate inverse N of B, then assembles

$$P = N^T D.$$

For each column k of N, the initial sparsity pattern is just J={k}. The algorithm forms a small reduced least-squares problem

$$\min_m \| B(I,J)m - e_k(I) \|_2,$$

where I is the shadow (the rows touched by columns in J). It solves that problem by Householder QR, not by normal equations. If the residual is too large, it scores candidate columns, adds at most beta=8 useful

candidates, and refits. The default maximum number of growth-and-refit events is alpha=lceil n/8rceil.

The construction target is a reduced residual no larger than epsilon, default 0.3. This makes P a compromise: a denser pattern may improve the approximation but costs more memory and work.

2. GMRES: compute one correction

Suppose the current solution estimate is x. Its original residual is

$$r = b - Ax.$$

Instead of solving directly for a new x, GMRES computes a correction d from the left-preconditioned system

$$PAd = Pr.$$

The implementation applies the operator as two sparse products,

$$v \mapsto Av \mapsto P(Av),$$

rather than materializing PA as a separate matrix.

GMRES builds a Krylov basis through Arnoldi iteration and modified Gram-Schmidt orthogonalization. It uses Givens rotations to solve the small least-squares problem. It is unrestarted, so it can use up to n iterations and can consume substantial memory for large n.

The inner stopping rule is the recomputed relative preconditioned correction residual:

$$\frac{\|Pr - PAd\|_2}{\|Pr\|_2} \leq \tau.$$

For FP64, tau=10^{-8}. The solver checks the computed residual before accepting; it does not rely only on GMRES's cheap Hessenberg estimate.

3. Iterative refinement: update the answer

The initial estimate is

$$x_0 = Pb.$$

Then the outer loop repeats:

- 1. Calculate r_i=b-Ax_i.
- 2. Use GMRES to approximately solve PAd_i=Pr_i.
- 3. Update x_{i+1}=x_i+d_i.
- 4. Evaluate the original-system backward error.

The final acceptance criterion is intentionally based on the original system, not only on the preconditioned one:

$$\eta(x) = \frac{\|b - Ax\|_\infty}{\|A\|_\infty \|x\|_\infty + \|b\|_\infty}.$$

For FP64, success requires eta(x)<=q10*2^{-53}. There are at most 20 outer corrections. This distinction matters: a small residual after preconditioning alone is not a sufficient certificate of the original answer.

End-to-end diagram

```
Sparse matrix A and RHS b
        |
        v
SPAI construction: build sparse P approximately equal to A^-1
        |
        v
Initial solution x0 = P b
        |
        v
Outer refinement: r = b - A x
        |
        v
GMRES: approximately solve P A d = P r
        |
        v
Update x = x + d; check original backward error
        |
        +-- success / another outer step / named failure
```

Literature position

The key research references are the Carson-Khan SPAI-GMRES-IR paper and the Khan-Carson BSPAI-GMRES-IR paper listed in PAPER_INDEX.md. The first gives the central SPAI-GMRES-IR setting. The second motivates BSPAI research.

Be precise in a presentation:

- Say the project is informed by those papers.
- Say this code implements a documented uniform FP64/FP32 baseline.
- Do not say it reproduces every published mixed-precision experiment.
- Do not say GPU SPAI is novel; prior GPU/MPI SPAI literature exists.
- Do not report a speedup unless you show recorded benchmark conditions and comparable measurements.

Files you should recognize

File - Why it matters to your explanation

docs/PHASE2A_ALGORITHM_SPEC.md - FP64 mathematical contract and project choices.

docs/EXPERIMENT2A_FP32_SPEC.md - Separate FP32 contract.

PAPER_INDEX.md - Starting point for research citations.

src/spai.cpp - Concrete SPAI realization.

src/gmres.cpp - Concrete GMRES realization.

src/ir.cpp - Concrete refinement realization.

Suggested speaking script (about 75 seconds)

"Our problem is to solve a sparse linear system Ax=b. Directly forming an inverse would generally be dense and expensive, so our solver first builds a sparse approximate inverse preconditioner P. This makes PA closer to the identity. We then use left-preconditioned GMRES to solve for a correction to the current solution, and iterative refinement repeatedly applies those corrections. GMRES stops using a preconditioned residual, while the overall solver accepts only when the original system has sufficiently small backward error. The implemented baseline is uniform FP64 or FP32; BSPAI and mixed precision are research directions rather than active implementation

claims."

Likely questions and safe answers

Implementation deep dive: from one equation to the program

Start with Ax=b. A is the given sparse matrix, b is the known right-hand side, and x is the unknown vector. A sparse matrix contains mostly zeros, so the program stores only nonzero entries in CSR format rather than storing every zero.

To make the test reproducible, the driver can choose a known x_true and compute b=A*x_true. The solver sees A and b, not the secret answer. After solving, validation can compare the returned solution and its residual against an independently computed reference.

Stage A: scaling and SPAI

An exact inverse would satisfy A^{-1}A=I, but an exact inverse is generally dense and expensive. SPAI searches for a sparse matrix P that behaves approximately like A^{-1}. In this project the useful target is that P*A behaves like the identity matrix.

Before the search, the implementation scales the problem. Scaling changes units so rows or columns with very different magnitudes do not dominate merely because their numbers are larger. It also forms the orientation used by the columnwise least-squares construction.

For each column of P, the algorithm performs this sequence:

- 1. Begin with a small pattern, meaning the permitted nonzero positions in that column.
- 2. Form a reduced least-squares problem containing only the rows and columns relevant to that pattern.
- 3. Solve it by QR factorization. QR is a stable way to solve least-squares problems without explicitly forming fragile normal equations.
- 4. Compute the fitted residual, which measures how far this column is from its target identity column.
- 5. Stop if the residual is small enough or the configured step limit is reached.
- 6. Otherwise form a shadow set of possible new positions, score the candidates, add the best permitted positions, and repeat.
- 7. Canonicalize the sparse output: sort indices, combine duplicates where applicable, and remove structural ambiguity.

The CUDA version preserves these mathematical stages. GPU kernels perform scaling, reduced-matrix gathering, fitted-residual evaluation, candidate scoring, and final sparse assembly. cuSOLVER performs QR, cuBLAS performs the triangular solve, and the CPU coordinates irregular pattern decisions.

Stage B: left-preconditioned GMRES

Iterative refinement needs a correction d satisfying

$$Ad = r,$$

where r=b-Ax is the current residual. The code instead solves the left-preconditioned problem

$$PAd = Pr.$$

If P is a useful approximate inverse, P*A is closer to the identity and GMRES can solve the correction problem more easily.

GMRES builds a Krylov basis. In ordinary language, it repeatedly applies the operator P*A to discover useful search directions. Arnoldi orthogonalization keeps new directions independent of earlier directions. The small coefficients form an upper-Hessenberg matrix H; "upper Hessenberg" means everything below the first subdiagonal is zero. Givens rotations update a small least-squares problem and provide an inexpensive residual estimate.

This project uses unrestarted GMRES. It does not throw away the basis after a fixed restart length. That matches the intended algorithm but can consume memory as the iteration count grows. The FP32 CUDA implementation mitigates needless up-front allocation by growing basis storage in 16-vector slabs.

The estimated residual is not the final authority. At exit, the code reconstructs d and explicitly recomputes

$$\rho = \frac{\|Pr - PAd\|_2}{\|Pr\|_2}.$$

This catches false convergence caused by rounding or a misleading recurrence. The current verification bound is 1e-8 in FP64 and 1e-4 in FP32.

Stage C: iterative refinement

The outer loop is deliberately easy to state:

```
build P once
choose x_0, using the preconditioner
repeat:
    compute r = b - A*x
    if the whole-solution backward error is small, return x
    use GMRES to approximately solve A*d = r
    if the correction failed verification, report failure
    update x = x + d
```

The preconditioner is not rebuilt during every outer step. The CUDA GMRES context also keeps device copies of A and P across corrections. The CPU currently stages the outer refinement decisions and computes the final whole-solution verification.

Research baseline versus project engineering

The general SPAI, GMRES, and iterative-refinement ideas are established numerical methods. The Carson-Khan work is the central BSPAI-GMRES-IR baseline for this project. GPU/MPI SPAI literature must be consulted before calling parallel construction novel.

Project-specific engineering includes the four uniform-precision variants, the present CPU/GPU division, the persistent FP32 GPU GMRES context, lazy 16-vector basis slabs, and fused CUDA SPAI residual/scoring work. These should be described as implementation choices or experimentally tested optimizations, not as new mathematical algorithms.

"BSPAI" means a block sparse approximate inverse. The active code is SPAI rather than a completed block-SPAI production solver. "Mixed precision" means deliberately using different precisions at different stages; the active variants are uniform FP32 or uniform FP64. These are future directions, not current results.

Parameters and what each one controls

- SPAI tolerance: how accurately each approximate-inverse column must fit its target before pattern growth can stop.
- SPAI maximum steps: maximum number of pattern expansions per column.
- Candidate limit: how many new structural positions may be admitted at a growth step.
- GMRES tolerance: target for the inner correction solve.
- GMRES iteration cap: safety limit on Krylov basis growth.

- IR tolerance: target for the outer backward error.
- IR step cap: maximum number of correction cycles.

Changing these parameters changes work, memory, and sometimes convergence. It is not valid to loosen a tolerance silently and then report only a faster runtime.

A professor-level explanation of success

Success has three parts. First, the algorithm returns an accepted status rather than a numerical breakdown. Second, an independent calculation finds a small backward error. Third, when performance is discussed, outputs and convergence behavior are compared before and after the optimization. A speedup alone is not numerical success.

The correct high-level conclusion is: the repository contains four validated uniform-precision SPAI-GMRES-IR variants, with CPU reference implementations and CUDA-accelerated construction/correction work. Current evidence supports correctness on the recorded suites and localized performance improvements on recorded workloads. It does not yet support claims of completed BSPAI, mixed precision, or universal GPU speedup.

Algorithm-to-function trace

This section tells the algorithm member exactly where each mathematical idea becomes code.

Algorithm action - CPU implementation - CUDA implementation

Parse sparse input - read_matrix_market in cpu.cpp - same host parser before GPU work

Canonical sparse storage - CsrMatrix::from_entries - same host CSR, then uploaded

SPAI driver - construct_spai - cuda_construct_spai / fp32_cuda_construct_spai

Fit one SPAI column - construct_column - host adaptive loop plus GPU slot work

Candidate score - evaluate_spai_score - scores or single_score kernel

Apply P*A - apply_left_preconditioned - two cuSPARSE SpMV calls

GMRES - left_preconditioned_gmres - context solve methods

IR diagnostics - evaluate_ir_diagnostics - CPU-staged diagnostic calculation

Fixed-P refinement - refine_with_preconditioner - CUDA refinement wrapper/context

Complete solve - spai_gmres_ir - native CUDA SPAI-GMRES-IR wrapper

What each top-level method consumes and produces

construct_spai(A, options) consumes a square, canonical, finite CSR matrix. options.epsilon controls accepted fit, alpha growth attempts, and beta candidate admission. It produces a SpaiResult, not just P, because status and construction history are part of the research evidence.

left_preconditioned_gmres(A,P,rhs,options,initial_guess) consumes the original matrix, approximate inverse, correction right-hand side, tolerance/cap, and optional starting correction. It produces a GmresResult. The returned solution is the correction d, not the final overall solution.

refine_with_preconditioner(A,P,rhs,options,reference) consumes a prebuilt P. It produces an IrResult; solution is now the final overall x. The optional reference is used only for diagnostics and does not guide convergence.

spai_gmres_ir(A,rhs,options,reference) combines construction and refinement. It produces the same IR result but fills its construction field with the SPAI evidence.

Function internals in algorithm order

CsrMatrix::multiply(x) loops over rows and their stored nonzeros and accumulates one output per row. transpose() swaps row/column structure while retaining a canonical order. These two primitives support almost every later stage.

construct_column(B,k,options) initializes the column pattern, creates its shadow, extracts a reduced problem, calls the QR least-squares helpers, measures the full residual, evaluates candidates, grows the pattern, and returns a SpaiColumn. construct_spai repeats this for all columns, assembles the inverse-like matrix, applies scaling, and returns the preconditioner.

apply_left_preconditioned(A,P,v) deliberately computes P*(A*v) as exactly two sparse products. It does not form and round a separate matrix product P*A.

left_preconditioned_gmres validates inputs, handles zero/easy exits, builds the Arnoldi basis, updates the small Givens-rotated least-squares problem, forms candidate corrections, verifies the actual preconditioned residual, and maps the outcome to a status.

evaluate_ir_diagnostics forms residual, 2-norm, infinity norm, normwise and componentwise backward errors, and optional forward error. refine_with_preconditioner uses these diagnostics for the outer stopping decision, calls GMRES only when needed, rejects an unsuccessful inner status, applies a finite update, and detects stagnation or the step limit.

Why not calculate A^{-1} directly?

The exact inverse is usually dense even when A is sparse. SPAI keeps only a selected sparse pattern, so it is cheaper to store and apply.

Why use left preconditioning?

The code solves the transformed correction equation PAd=Pr. If P were an exact inverse, the operator would be the identity.

Why both GMRES and iterative refinement?

GMRES produces one approximate correction. Refinement uses the original residual after that correction to improve the whole solution iteratively.

Is the project a BSPAI implementation?

No. The project investigates the literature around BSPAI, but the active code implements adaptive SPAI with uniform FP32 or FP64 arithmetic.

What proves convergence?

The solver reports convergence only after the original-system normwise backward error meets the precision-specific threshold.